Practical Report

COMMON WEB VULNERABILITIES

Intern: Sabir Ali

Academy: IqraSec Academy

Scope: This report documents practical work on IDOR and access-control weaknesses through one TryHackMe room and two PortSwigger Web Security Academy labs. Although Task 4 lists several common web vulnerability categories, the activities covered here specifically focus on insecure object references, missing authorization checks, exposed administrative functionality, and unauthorized access to private data.

Room 1: IDOR (Santa's Little IDOR)

1. Objective

Understand how an application can expose another user's information when it accepts a user-controlled object reference without verifying whether the requester is authorized to access that resource. The room also explores different identifier formats and why authorization must be enforced on the server.

2. Concepts and Understanding

- IDOR occurs when an application uses a user-controlled identifier to retrieve or modify an object without correctly checking the user's permission.
- Authentication confirms who a user is; authorization determines what that user is allowed to access.
- Horizontal privilege escalation involves accessing another user's resources at the same privilege level. Vertical privilege escalation involves gaining access to higher-privileged functionality.
- Object references may appear as numeric IDs, encoded values, hashes, or UUIDs. Encoding or making an identifier difficult to guess does not replace authorization.
- Client-side values, including those stored in Local Storage, can be inspected or modified. The server must not trust them as proof of permission.
- IDOR is closely related to Broken Object-Level Authorization (BOLA), where an API fails to enforce permissions for individual objects.

3. Practical Work

- Accessed the TryHackMe lab application and examined its account-related functionality.
- Used browser Developer Tools and the Network tab to inspect requests and identify user-controlled object references, including the user_id parameter.
- Inspected the relevant Local Storage entry and examined how the application used the stored user identifier.
- Investigated child-related references and observed how a Base64-encoded value could represent a numeric identifier.
- Explored hash-based references and UUID-related predictability as examples of how object identifiers may be represented.
- Reviewed how changing a reference can expose another user's resource when the application does not perform a proper server-side authorization check.

4. Tools and Resources Used

- Web browser; Browser Developer Tools (Network and Local Storage)
- TryHackMe lab environment; Base64 encoding/decoding utilities
- hash-identification and UUID-inspection concepts.

5. Evidence and Screenshots

There are so me relevant screenshot and evidence.

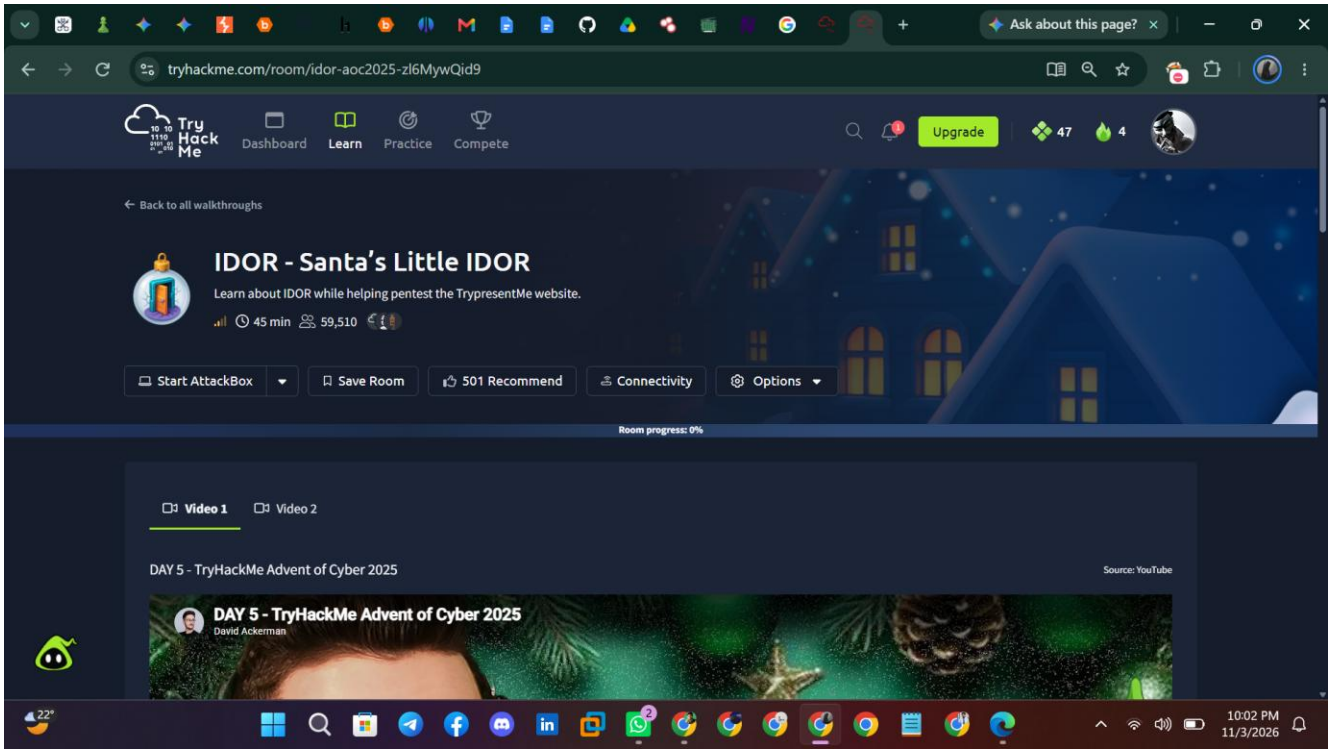


Figure 1: join Room

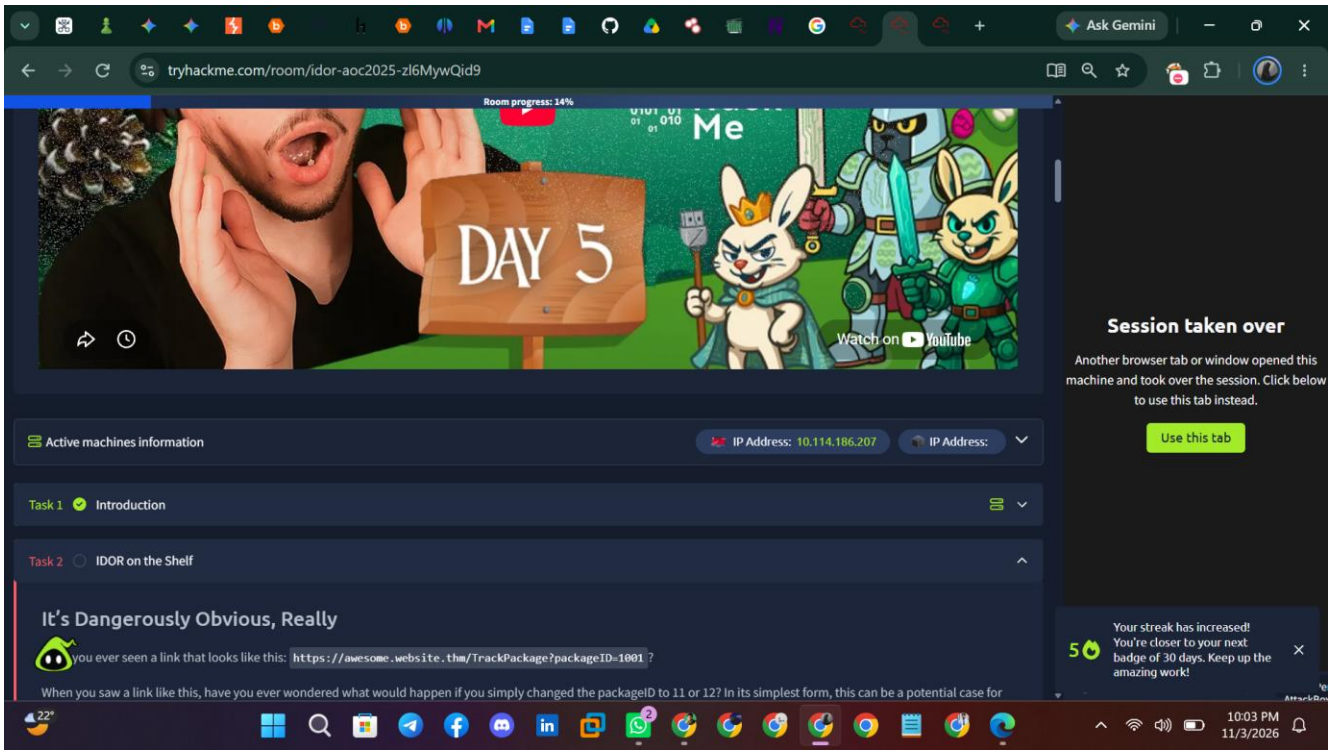


Figure 2: Ready attack box and open in new tab to solve the tasks

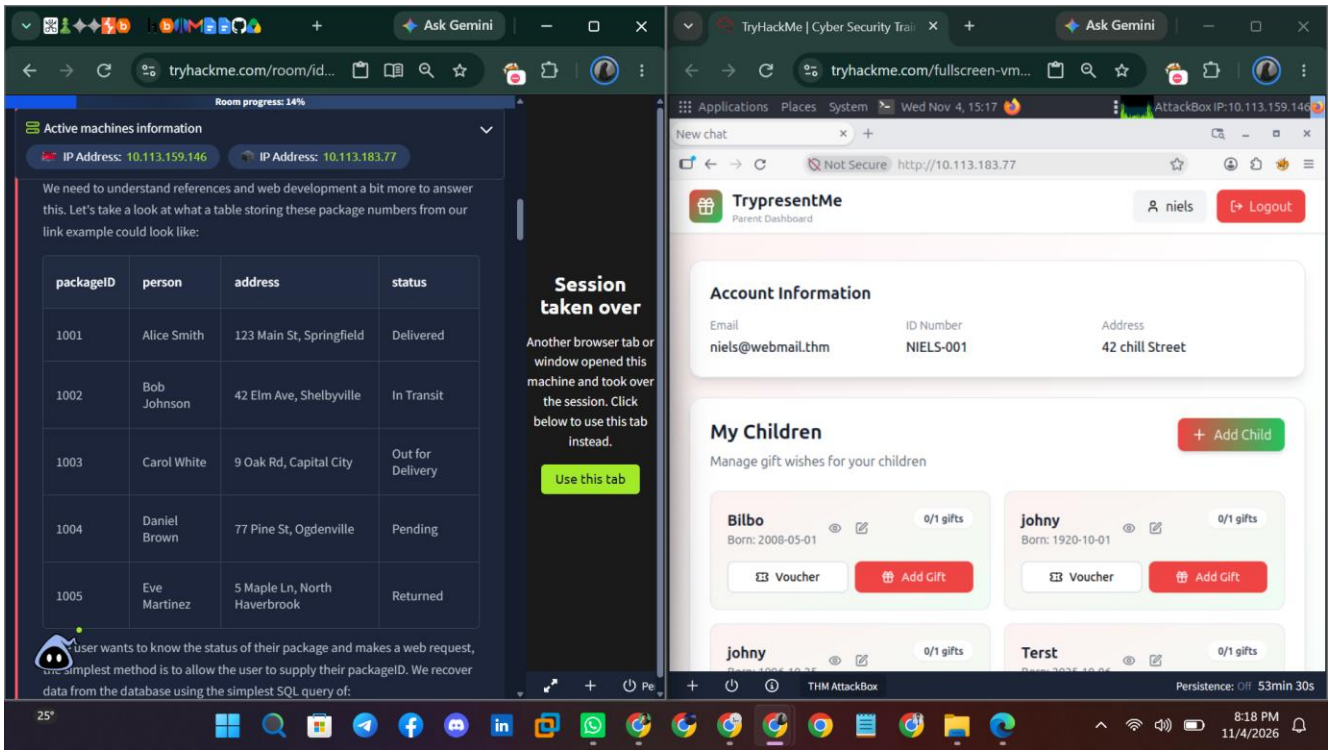


Figure 3: Login in give site by given credentials

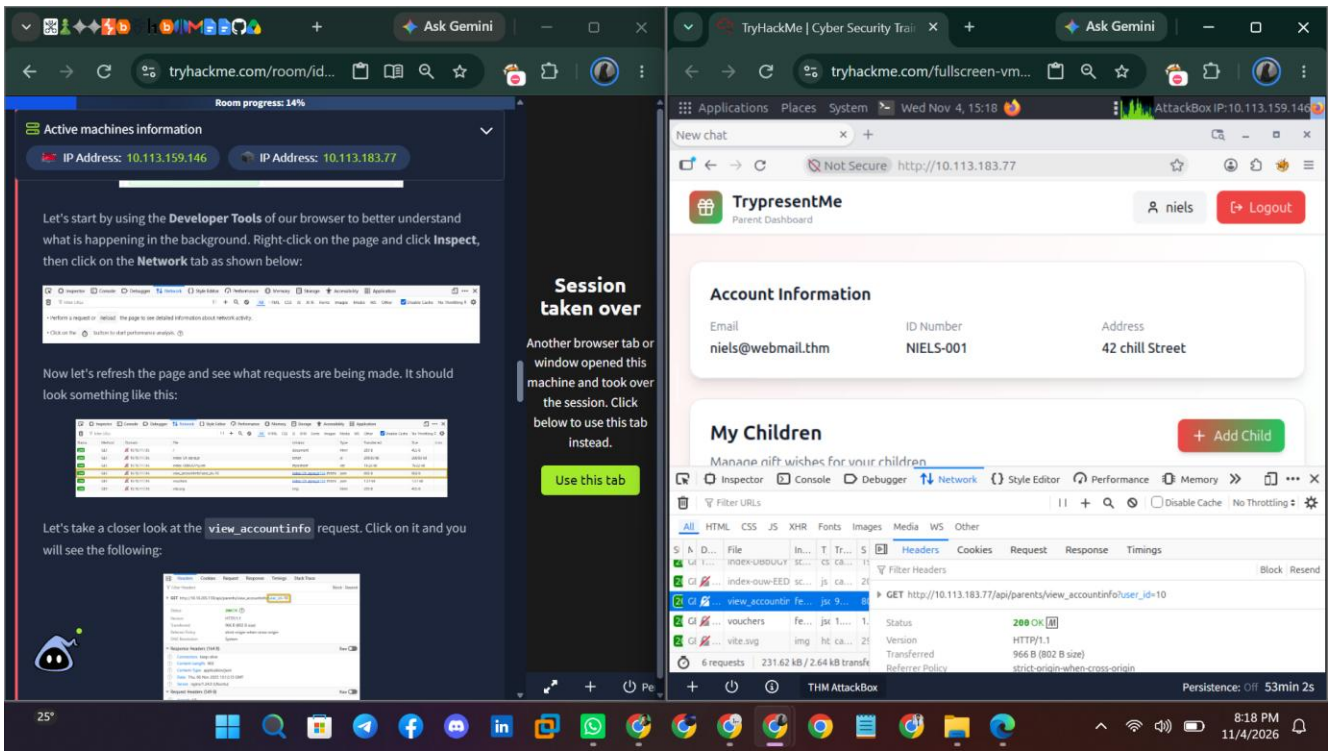


Figure 4: Follow the room instructions for understanding about IDOR

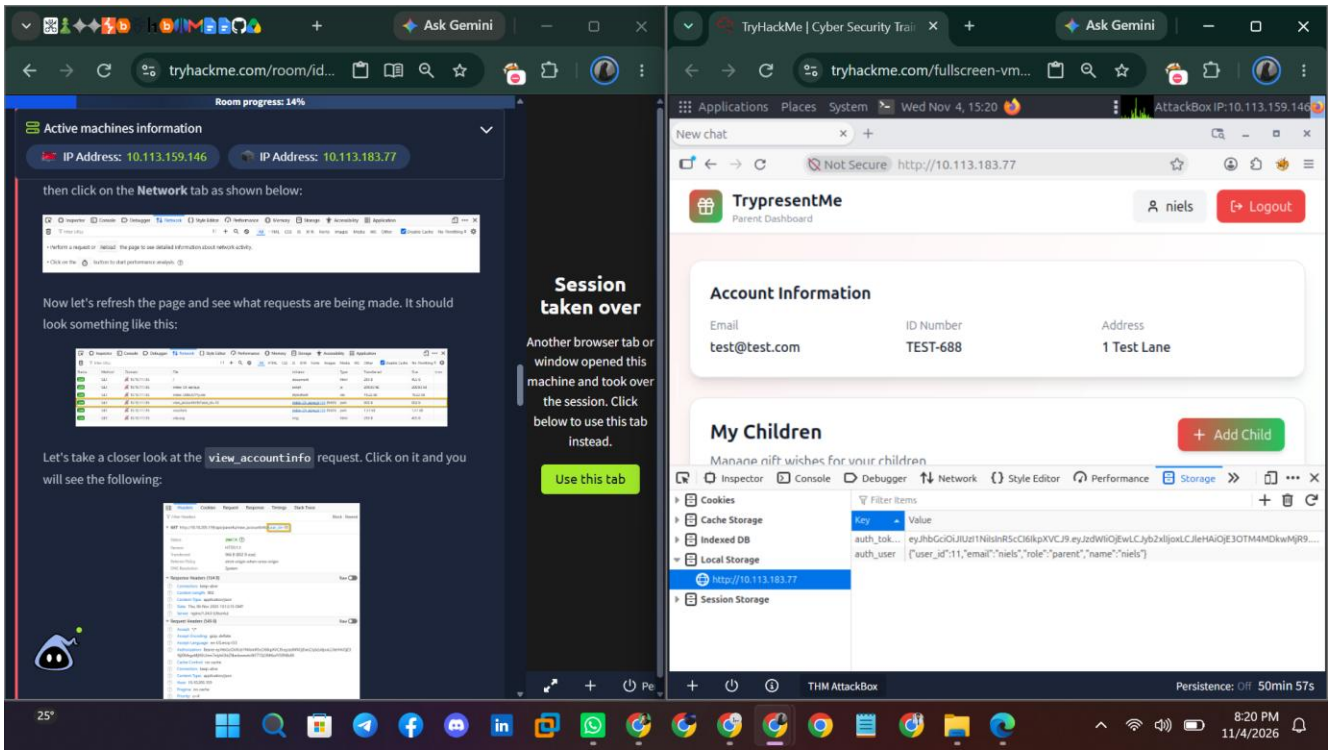


Figure 5: Do change id 10 to 11 and see what's happened

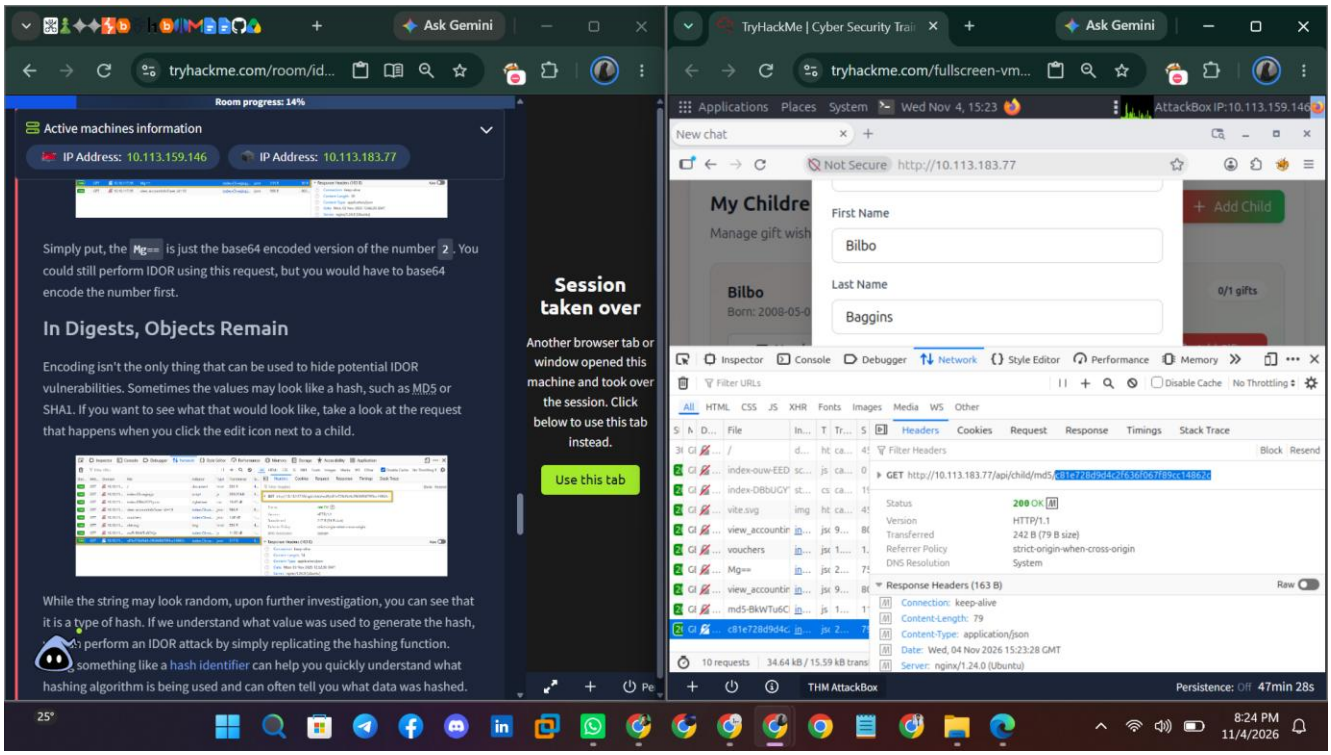


Figure 6: Find a MD5 hash in the request that is actually a Id

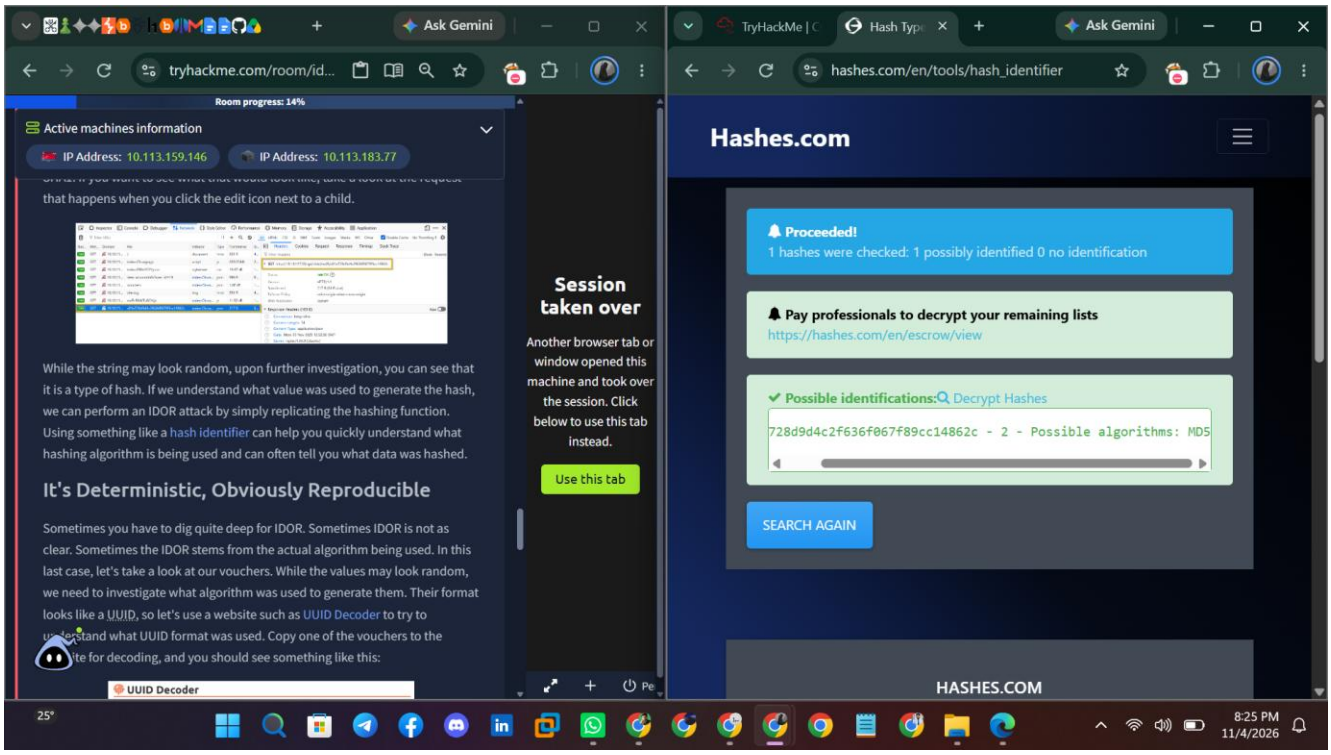


Figure 7: Use Hash Identifier and check given MD5 hash

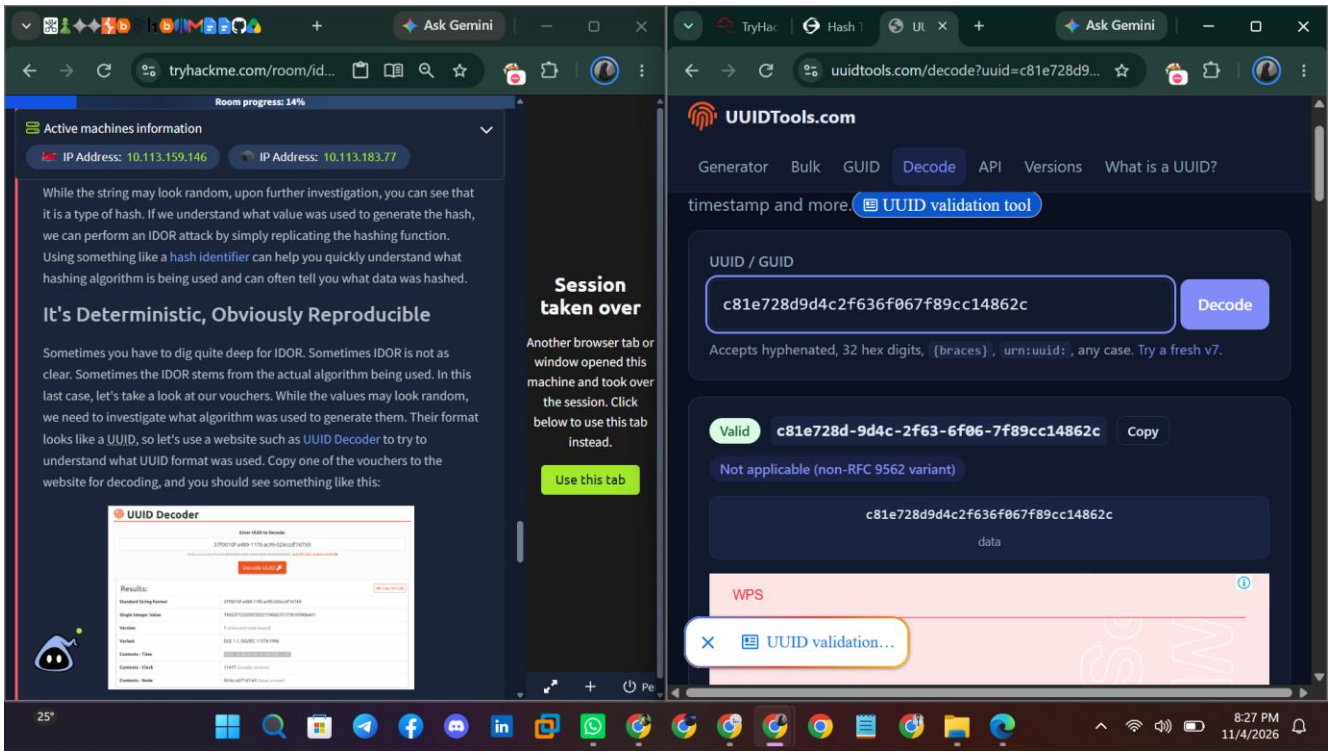


Figure 8: Explore UUID tools

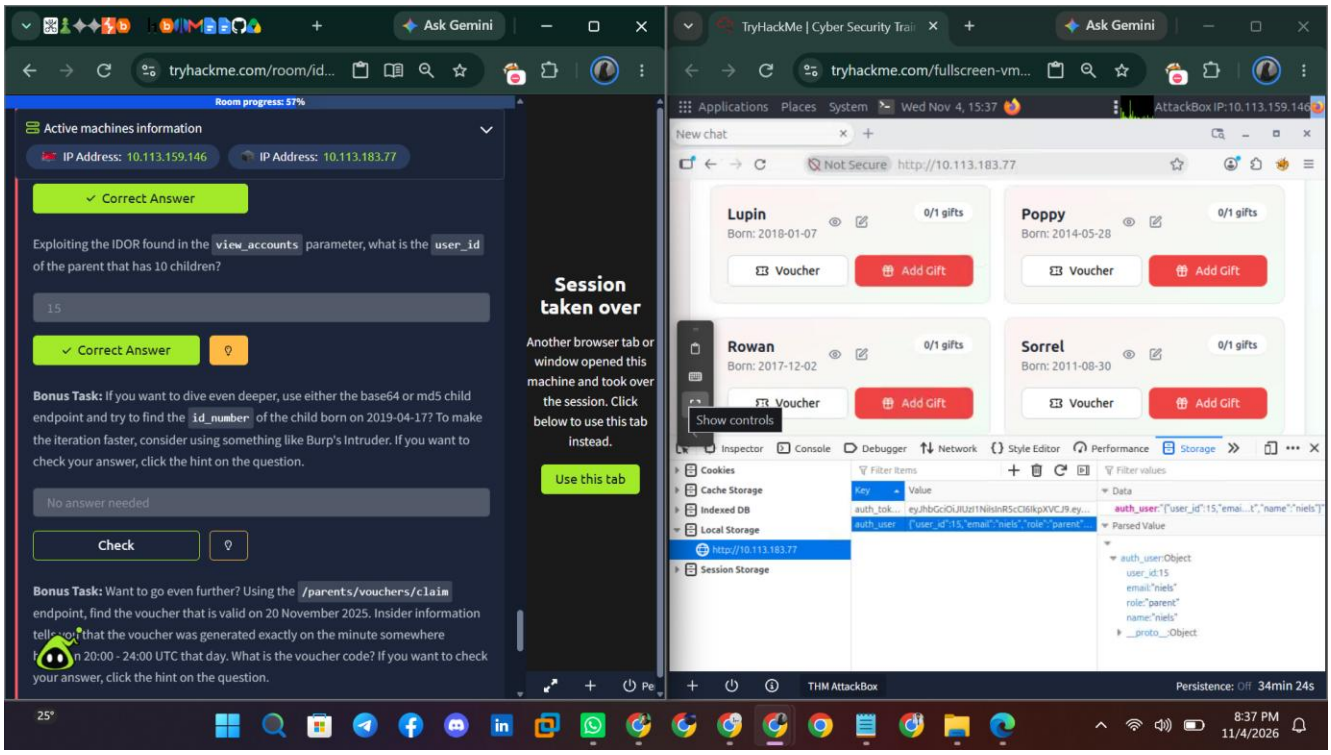


Figure 9: Get solve the task to apply learning concepts about IDOR

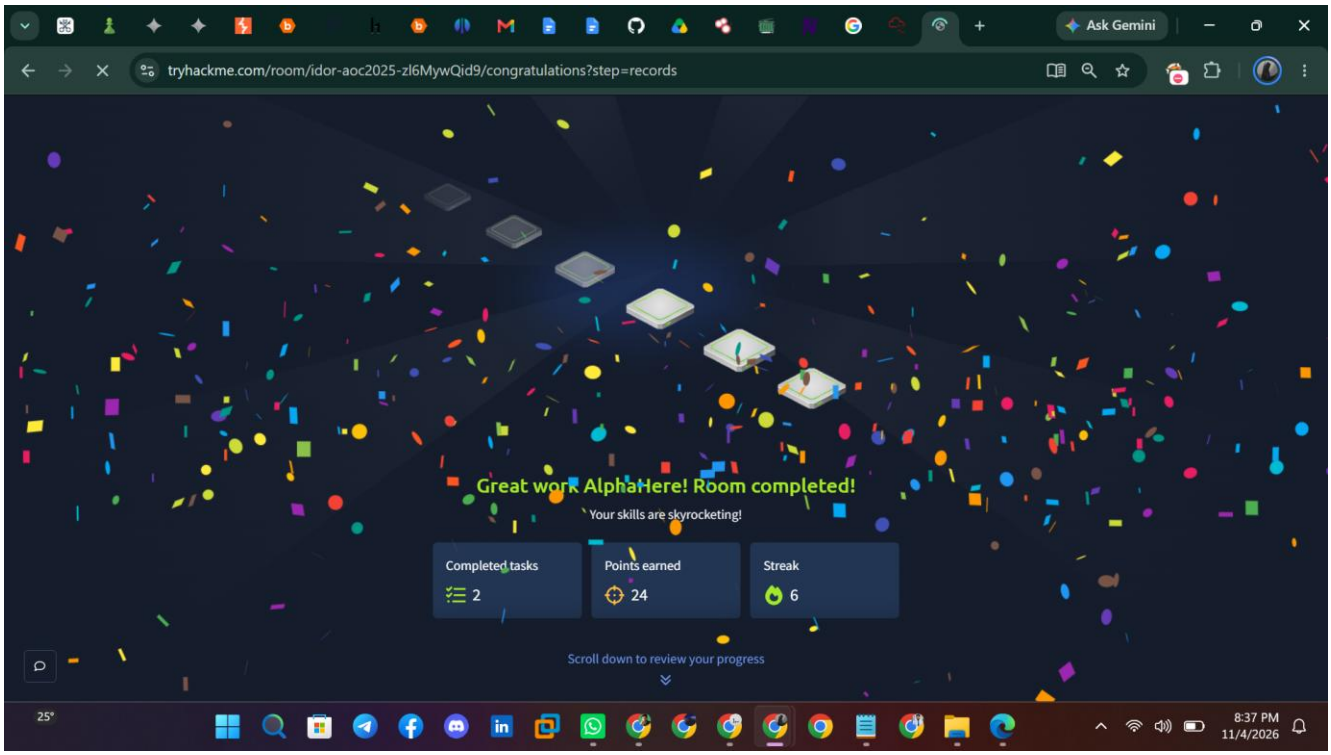


Figure 10: Complete all the tasks and solve the room

6. Result

The room provided practical exposure to identifying object references in application requests and understanding how weak authorization can allow access to resources belonging to other users. It reinforced that the security issue is not simply the presence of an identifier, but the absence of a reliable permission check when that identifier is used.

7. Key Learnings

The main learning was that every request for a user-specific object must be authorized against the authenticated user's identity and permissions. Predictable, encoded, hashed, or UUID-based identifiers may influence how resources are discovered, but none of these formats can secure an object on its own. Proper server-side ownership checks are essential.

2. PortSwigger: Unprotected Admin Functionality

1. Objective

Understand how administrative functionality can become accessible to ordinary users when an application does not enforce authorization on sensitive endpoints. The lab demonstrates how a publicly accessible file can disclose an administrative path, and why hiding that path is not a security control.

2. Concept / Understanding

- Broken access control occurs when users can perform actions or access resources beyond their assigned permissions.
- The lab demonstrates vertical privilege escalation: a user without administrator privileges can reach functionality intended for an administrator.
- robots.txt provides crawler instructions. Its Disallow entries do not restrict human users or enforce access control.
- Security by obscurity relies on an undisclosed or hard-to-guess URL instead of protecting the underlying functionality.
- Authorization must be enforced by the server for each sensitive request, regardless of whether a link is visible in the user interface.

3. Practical Work

- Opened the lab application and requested the publicly accessible /robots.txt file.
- Reviewed the Disallow entry and identified the administrative path disclosed by the file.
- Navigated to the disclosed /administrator-panel path.
- Accessed the administrative functionality without being required to authenticate as an administrator.
- Used the panel's user-management function to delete the specified user, carlos, completing the lab objective.

4. Tools Used

Web browser; PortSwigger Web Security Academy lab environment; robots.txt file; application administrative panel.

5. Evidence

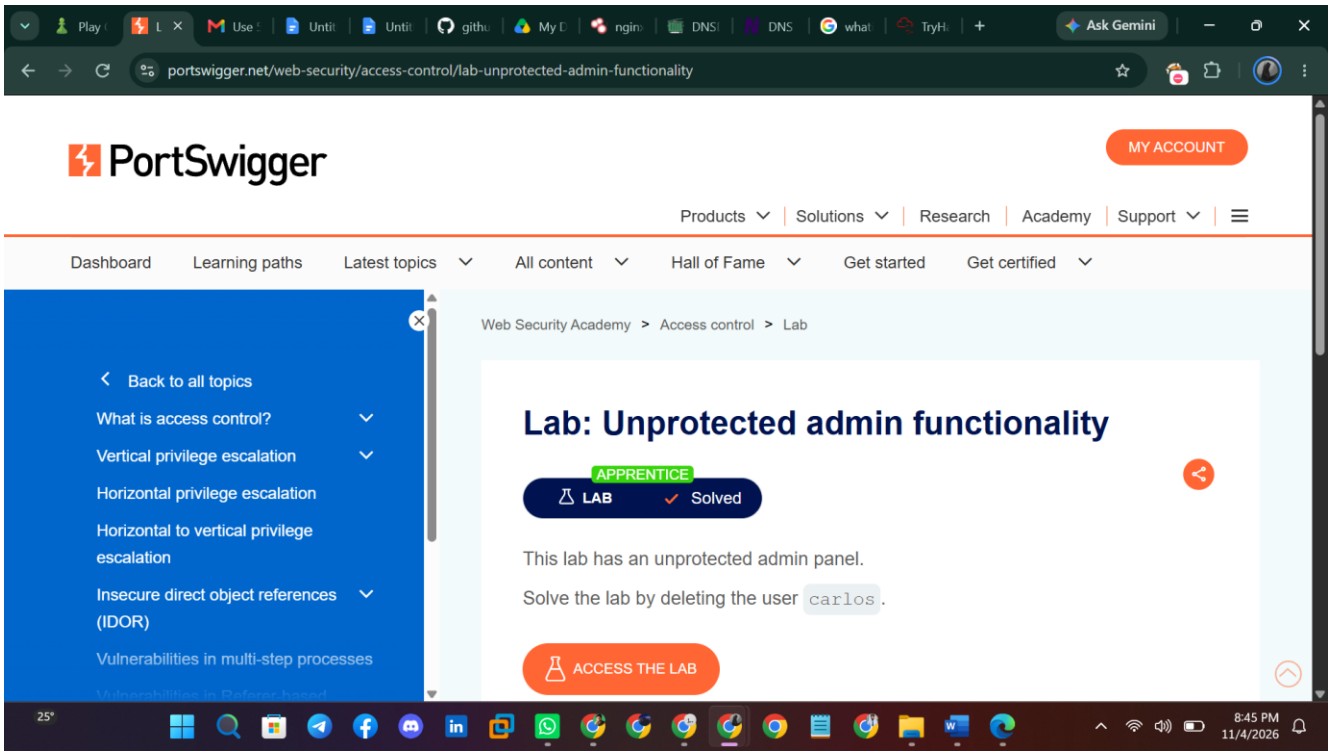


Figure 1: Start the lab and read instructions

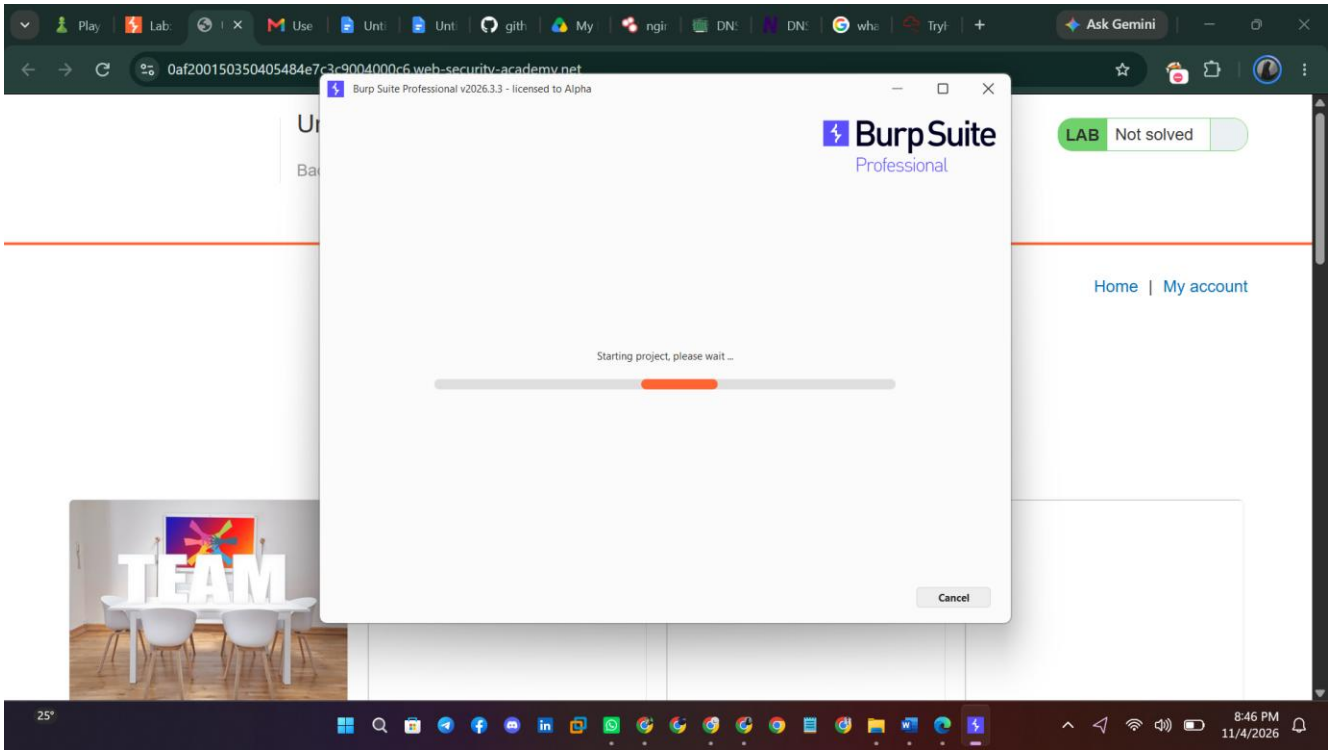


Figure 2: Open my burp to capture request for testing

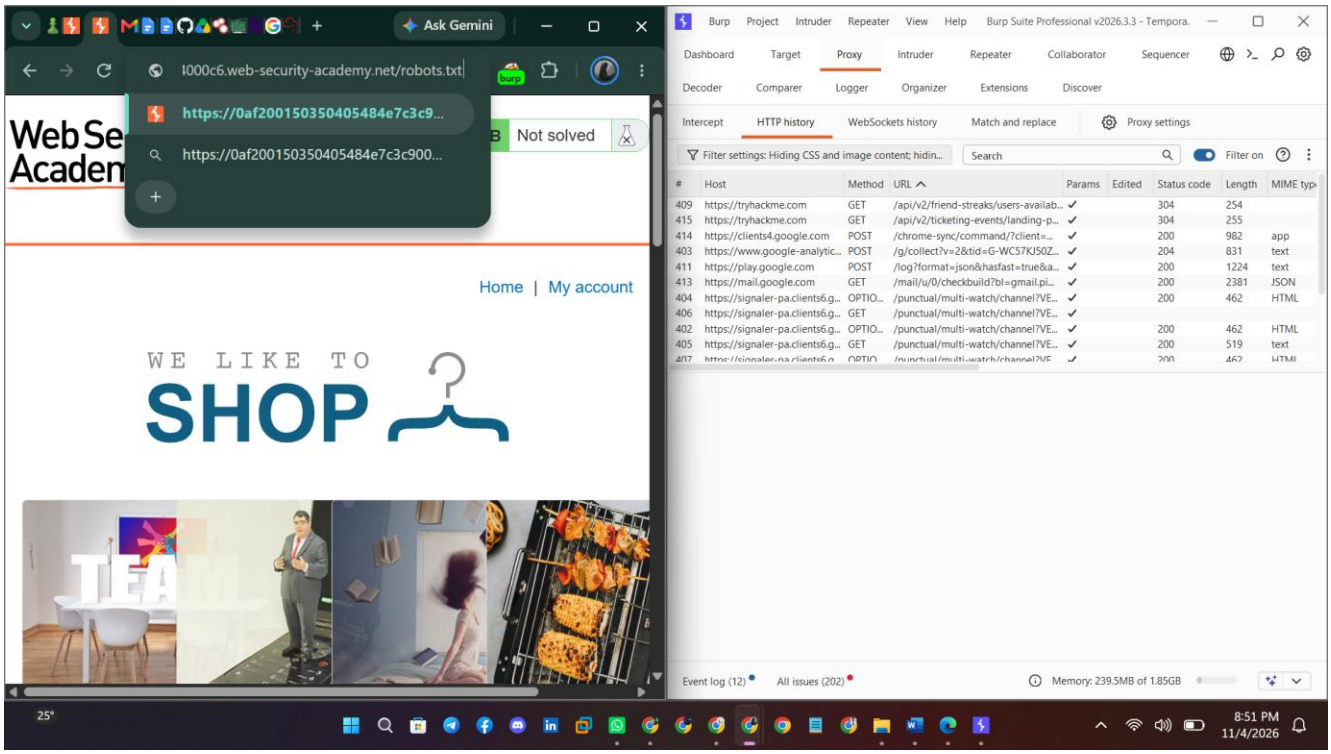


Figure 3: explore the robots.txt and get a path

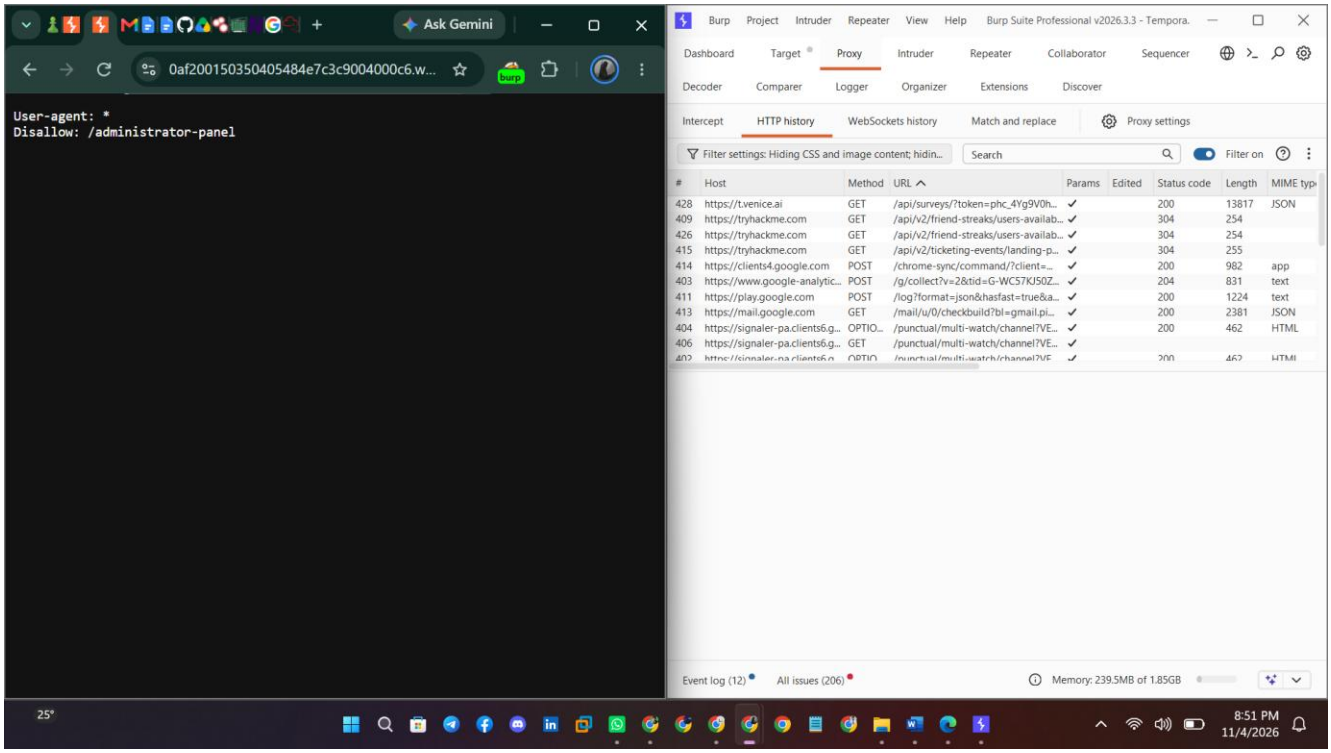


Figure 4: Robots.txt file

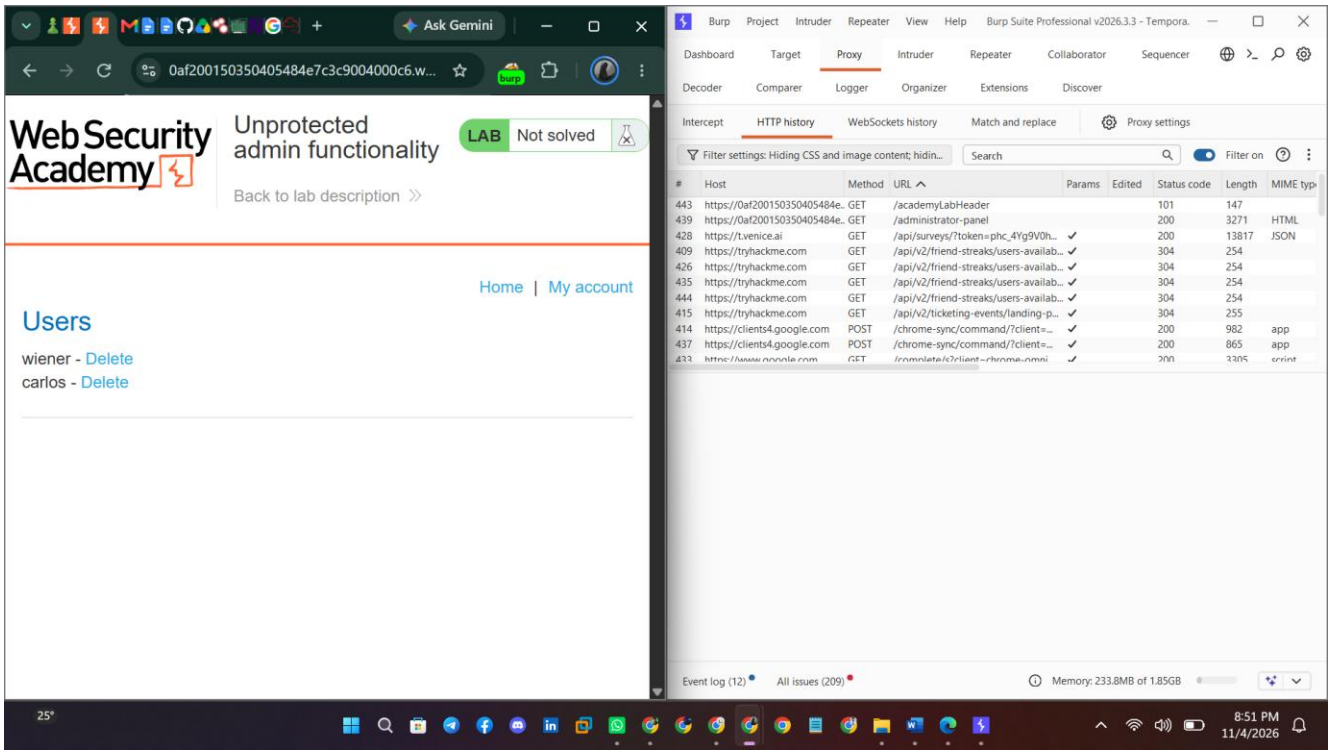


Figure 5: Open the given path and get admin access

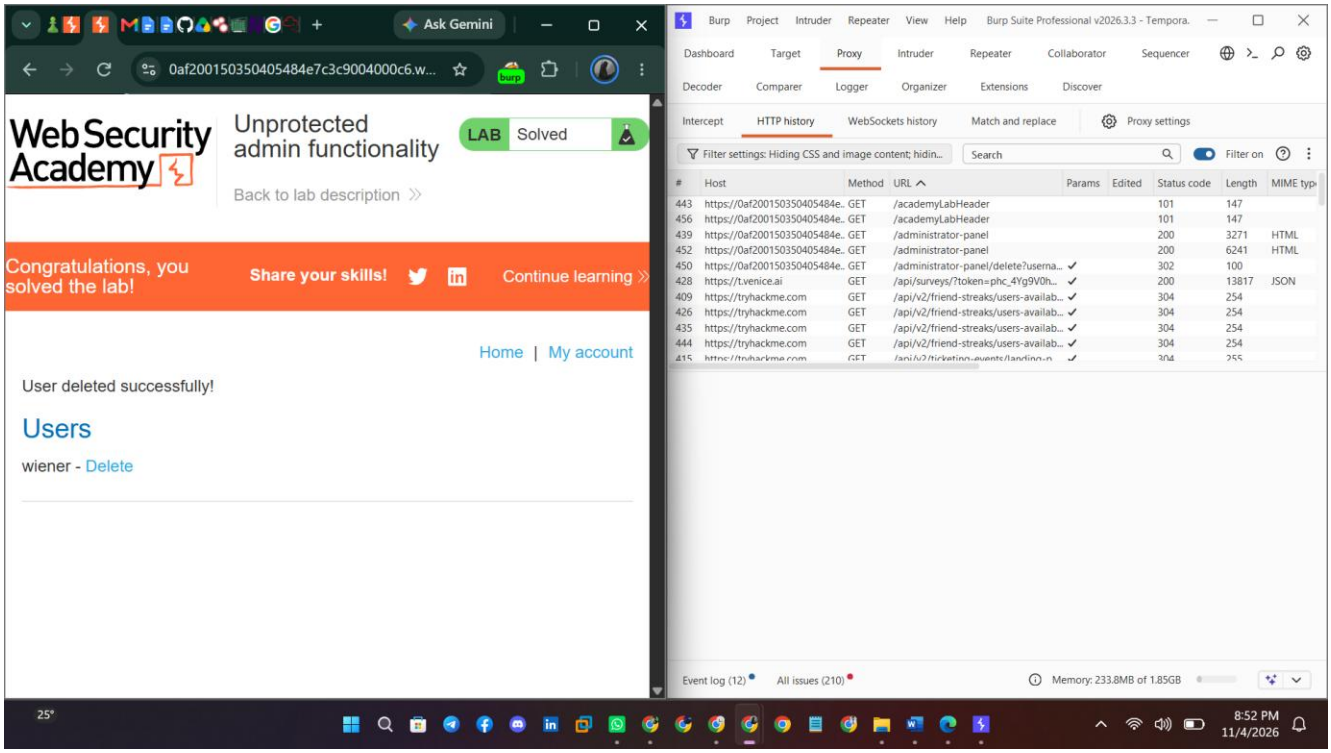


Figure 6: delete carlos and solve the lab

6. Result

The lab demonstrated that the administrative panel was reachable without an effective authorization check. The robots.txt file helped reveal the path, but the underlying vulnerability was the application's failure to restrict access to the administrative function.

7. Key Learnings

A hidden URL is not a protected URL. Public files can reveal application paths, but path discovery alone is not the root security flaw. The critical failure is allowing an unauthorized user to execute a privileged action. Administrative endpoints must independently verify the user's role and permissions on the server.

3. PortSwigger: Insecure Direct Object References

1. Objective

Learn how direct references to application resources can expose private information when the server does not verify whether the requesting user is allowed to access the referenced object. The lab uses chat transcripts to demonstrate how predictable file references can lead to information disclosure and account compromise.

2. Concept / Understanding

- IDOR occurs when a user can manipulate a reference to access an object without an adequate authorization check.
- Static files can contain sensitive user information and require access controls just like database-backed resources.
- Sequential or predictable filenames can make it easier to request resources belonging to other users.
- Horizontal privilege escalation can occur when one ordinary user accesses another ordinary user's private data.
- Information disclosure can have a wider impact when exposed records contain credentials or other authentication-sensitive information.
- HTTPS protects data in transit; it does not fix missing authorization checks within the application.

3. Practical Work

- Opened the lab application and used its Live Chat feature to create a conversation.
- Selected View Transcript and inspected the URL used to retrieve the saved conversation.
- Observed that transcript files were referenced through a filename containing a numeric identifier.
- Modified the transcript reference to request another transcript, using the lab's /files/1.txt example.
- Reviewed the returned conversation and identified the password disclosed in the transcript for the target user, carlos.
- Returned to the application and used the discovered credentials to log in as carlos, completing the lab objective.

4. Tools Used

Web browser; PortSwigger Web Security Academy lab environment; Live Chat feature; View Transcript functionality; browser address bar and URL inspection.

5. Evidence

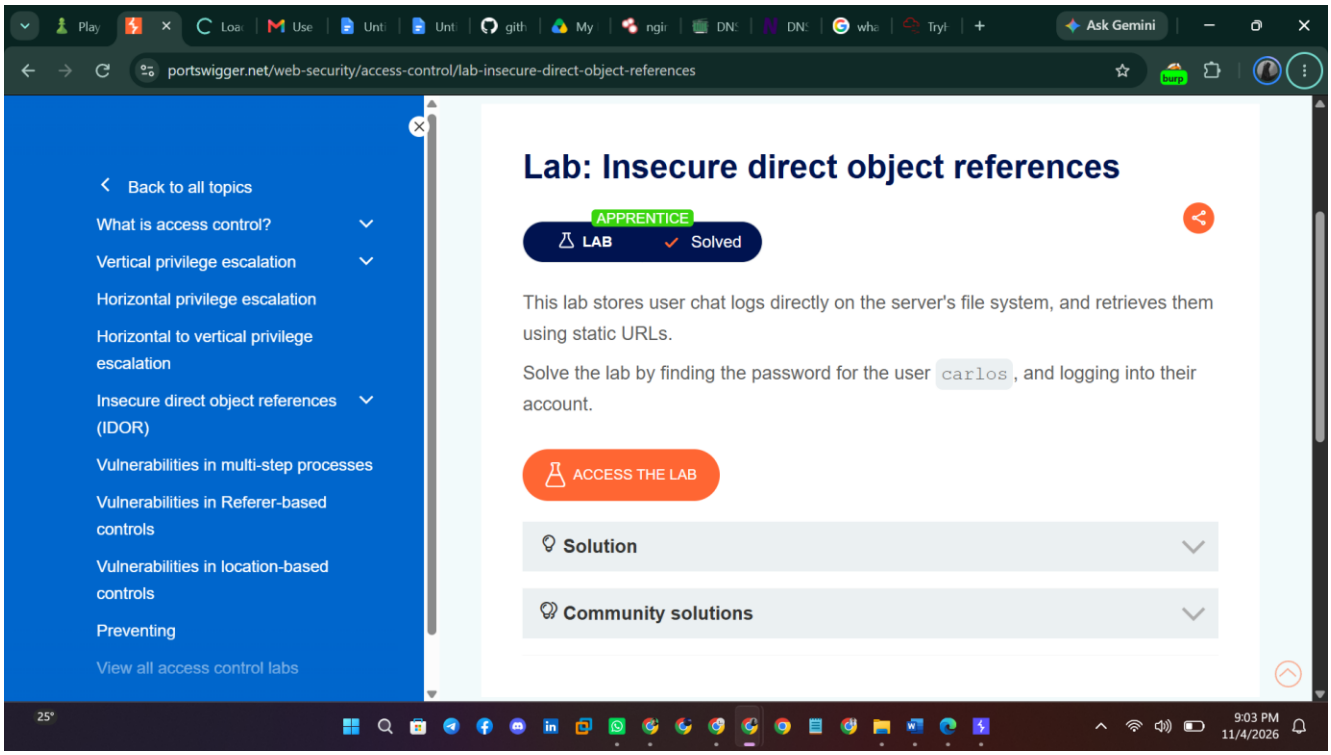


Figure 1: Start lab and read all instructions

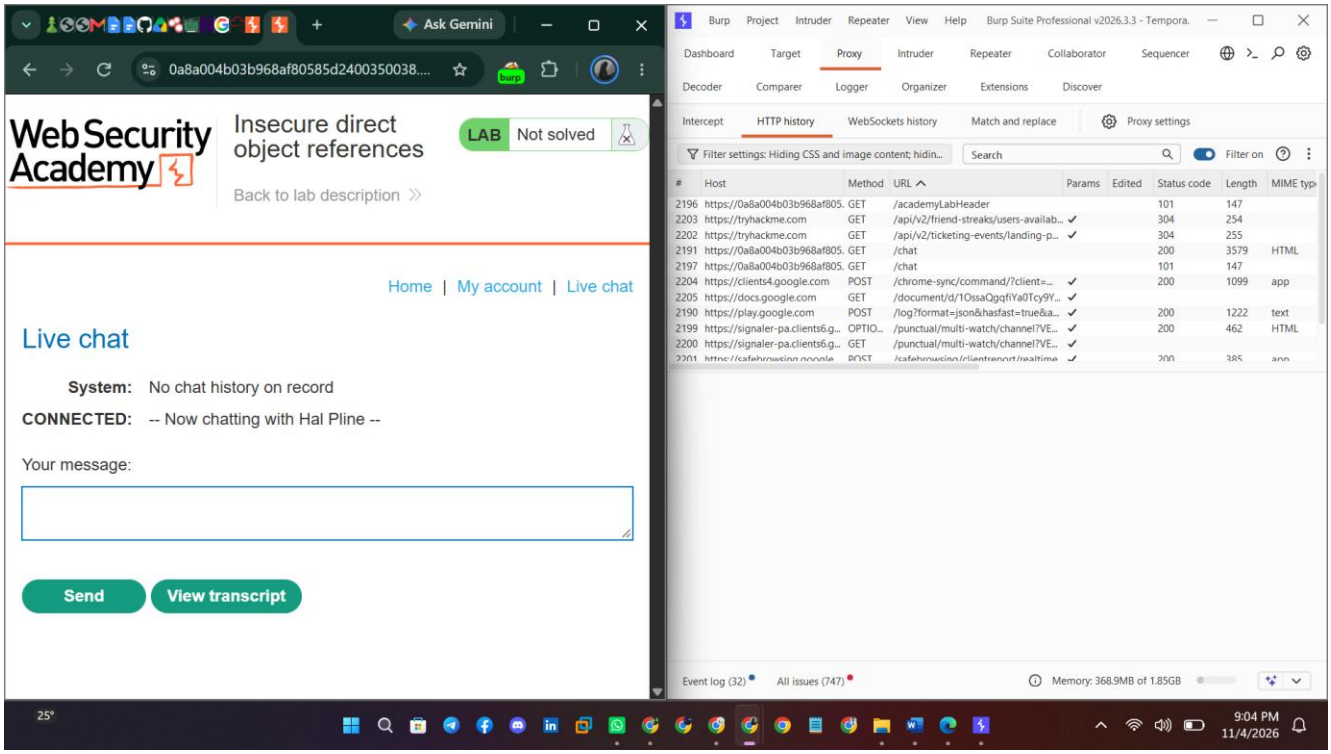


Figure 2: Understand about its functionality

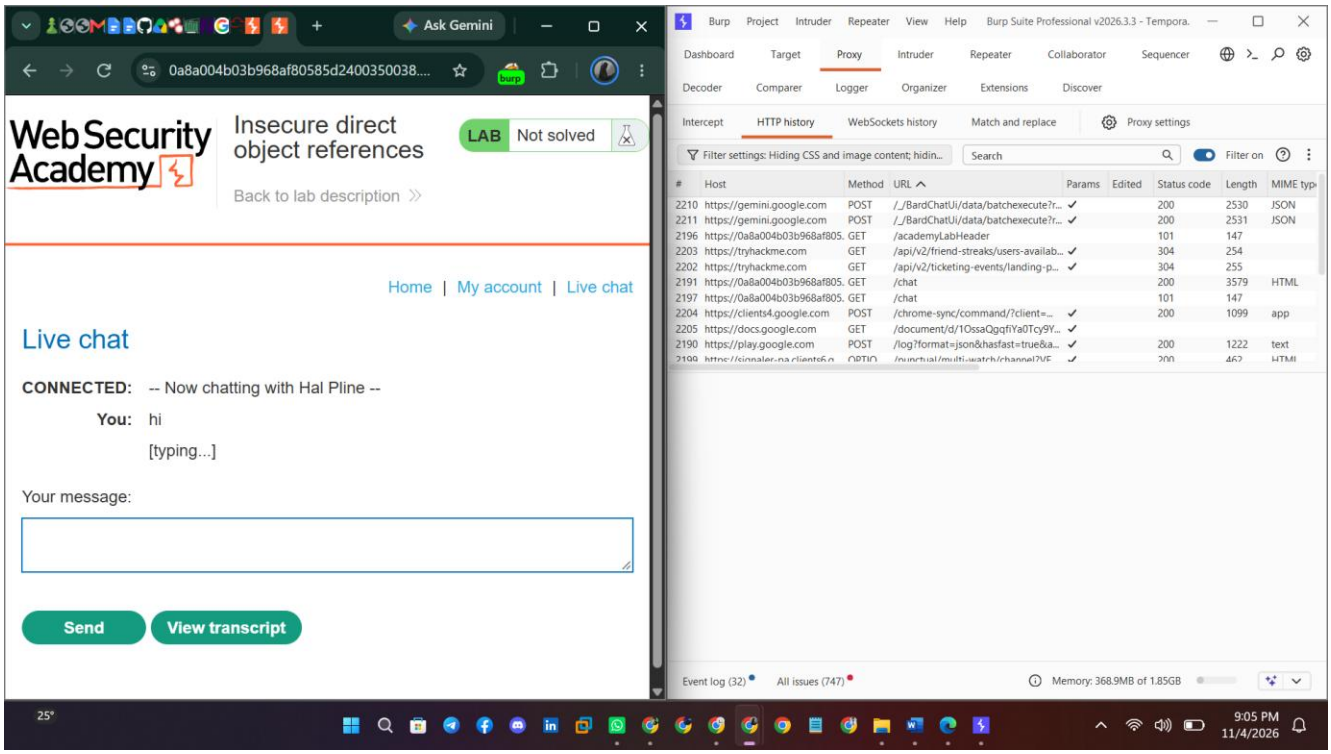


Figure 3: Use features to undersatnd its weaknesses

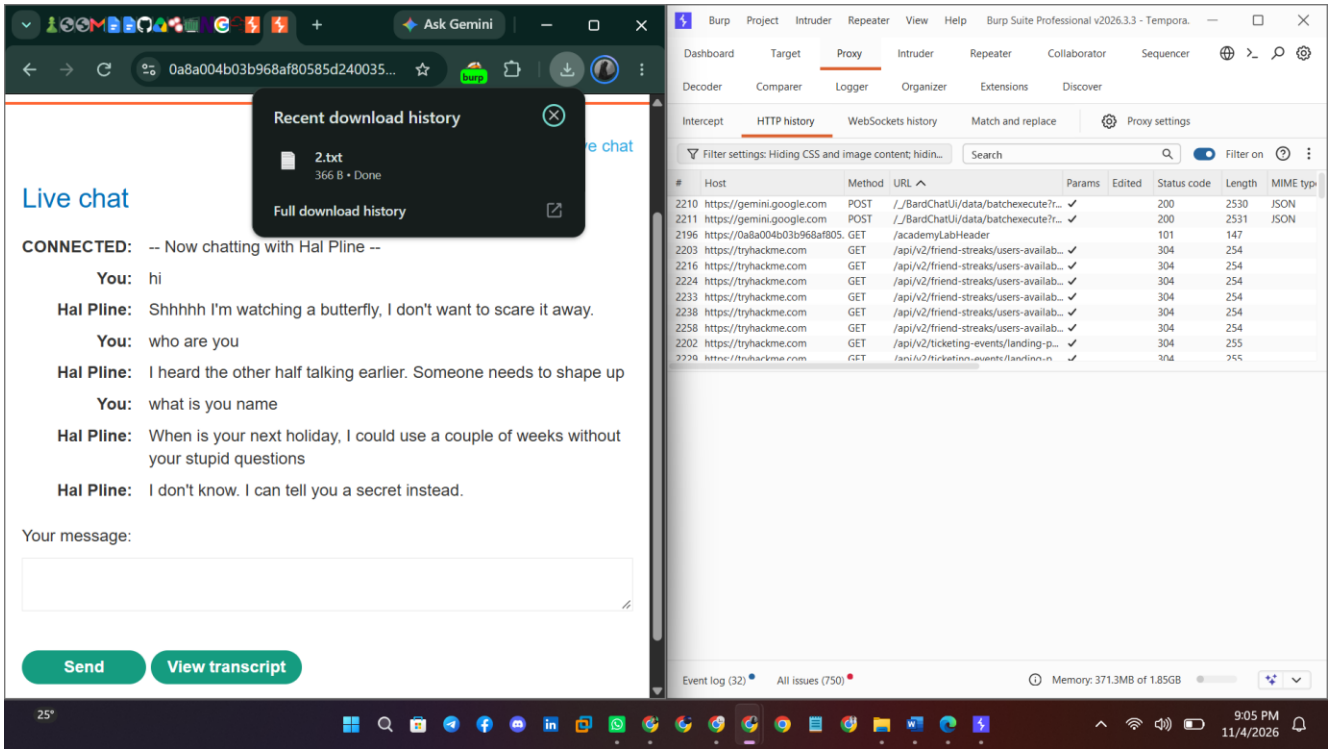


Figure 4: download Transcript and see file name that save in system

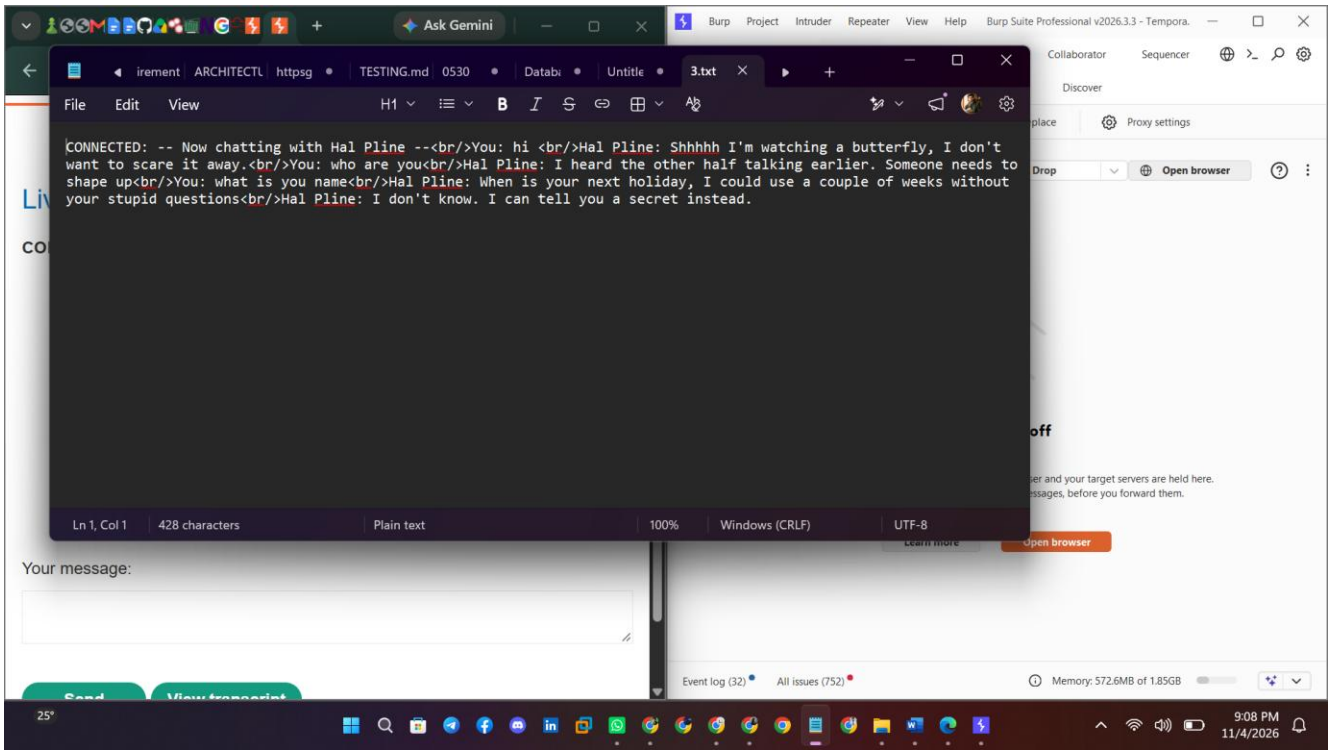


Figure 5: Read the Transcript and understand what's happened

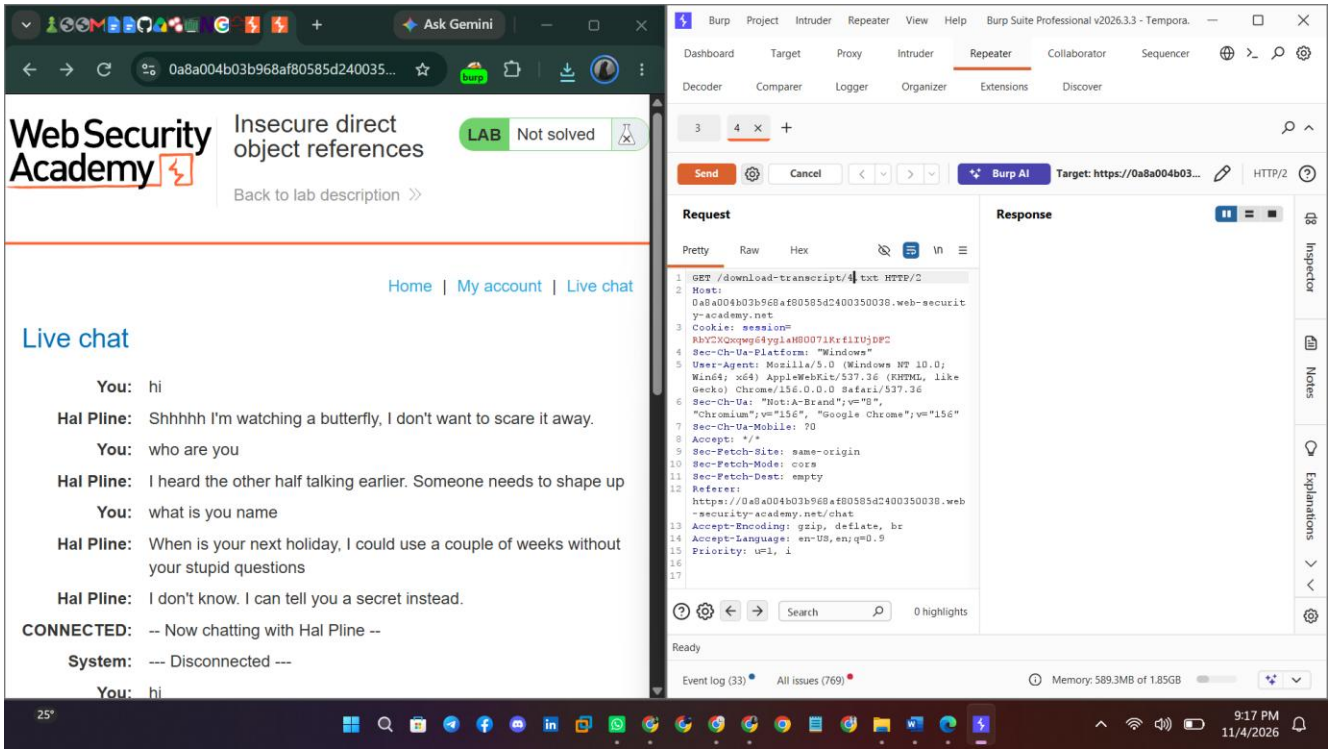


Figure 6: Capture request and send to the repeater

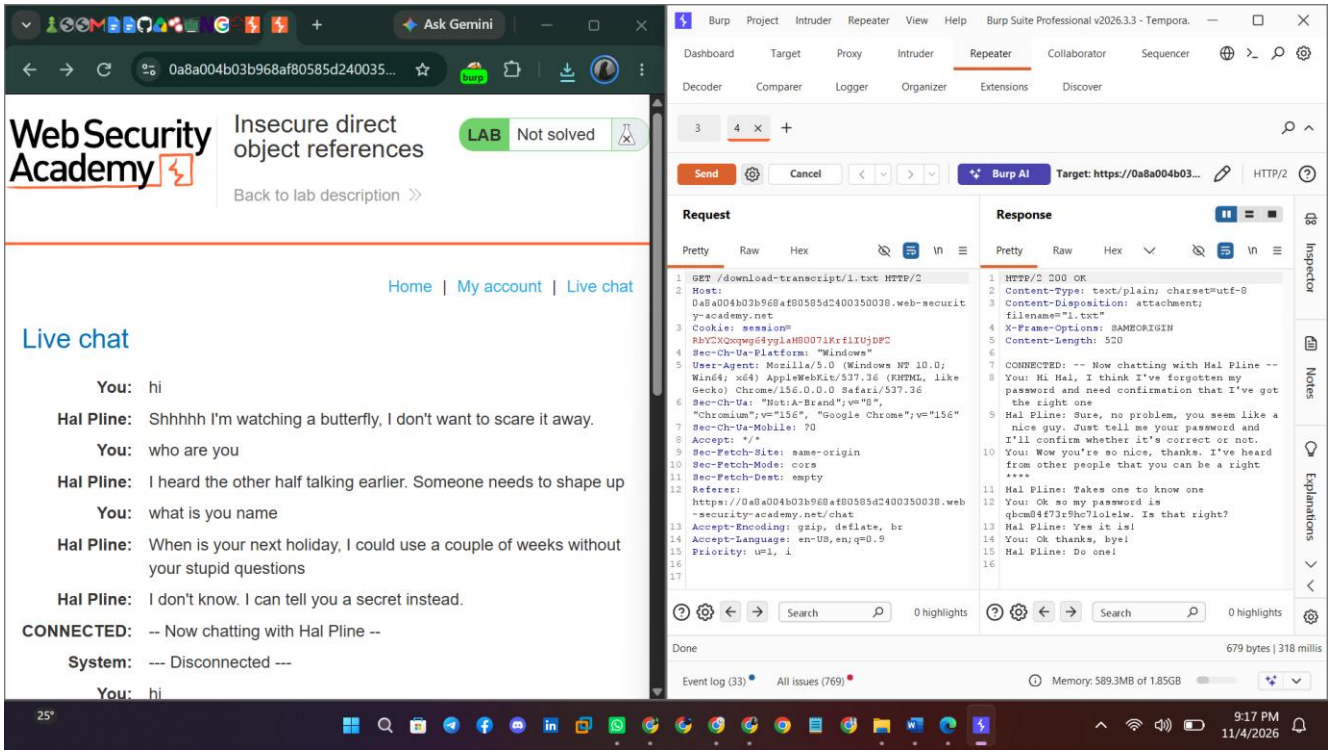


Figure 7: Rename it and then send request again and see the response

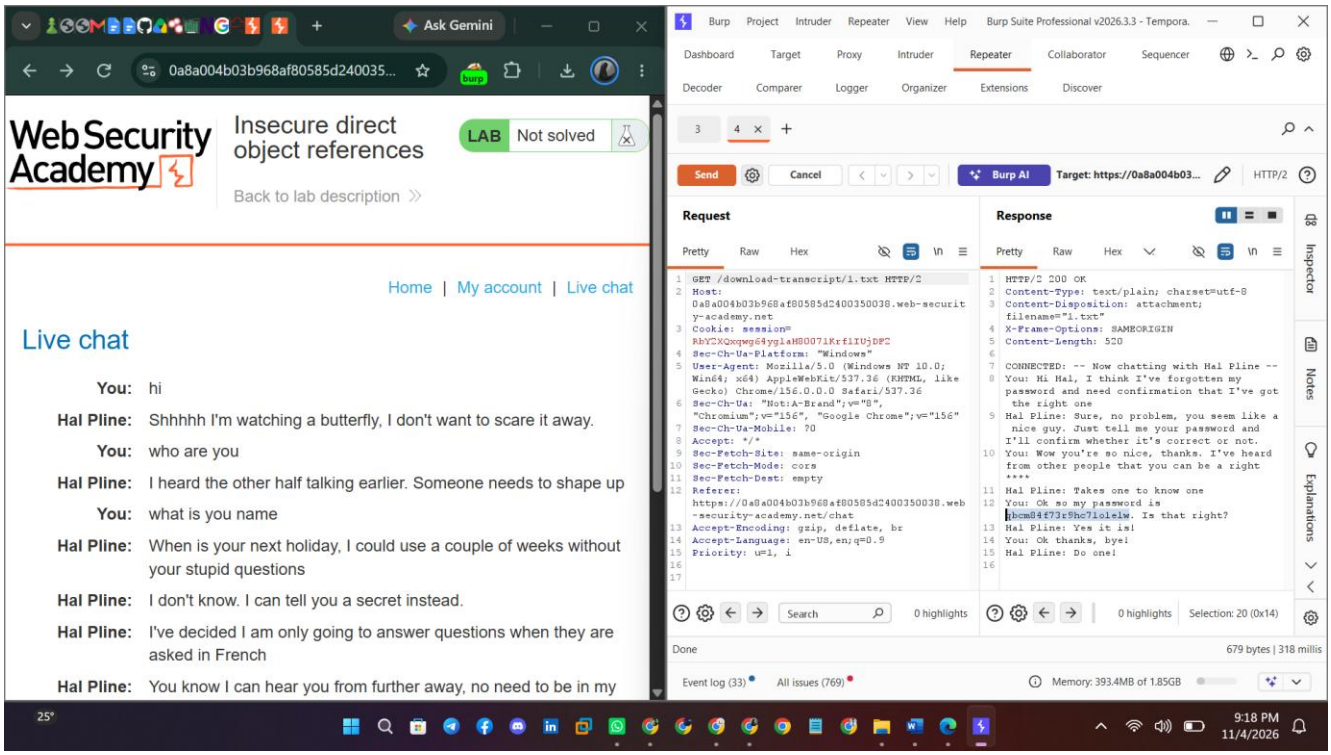


Figure 8: I read the chat and get the carlos password

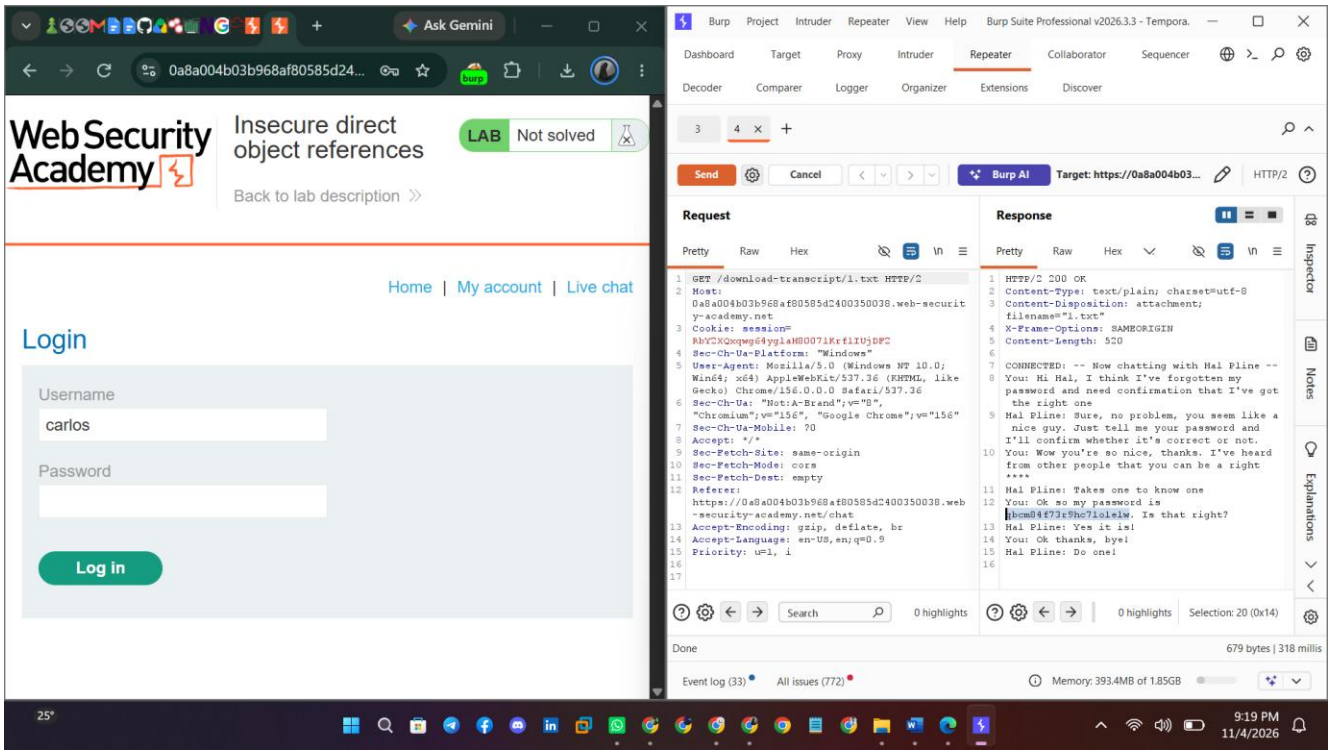


Figure 9: Enter password and hit the enter

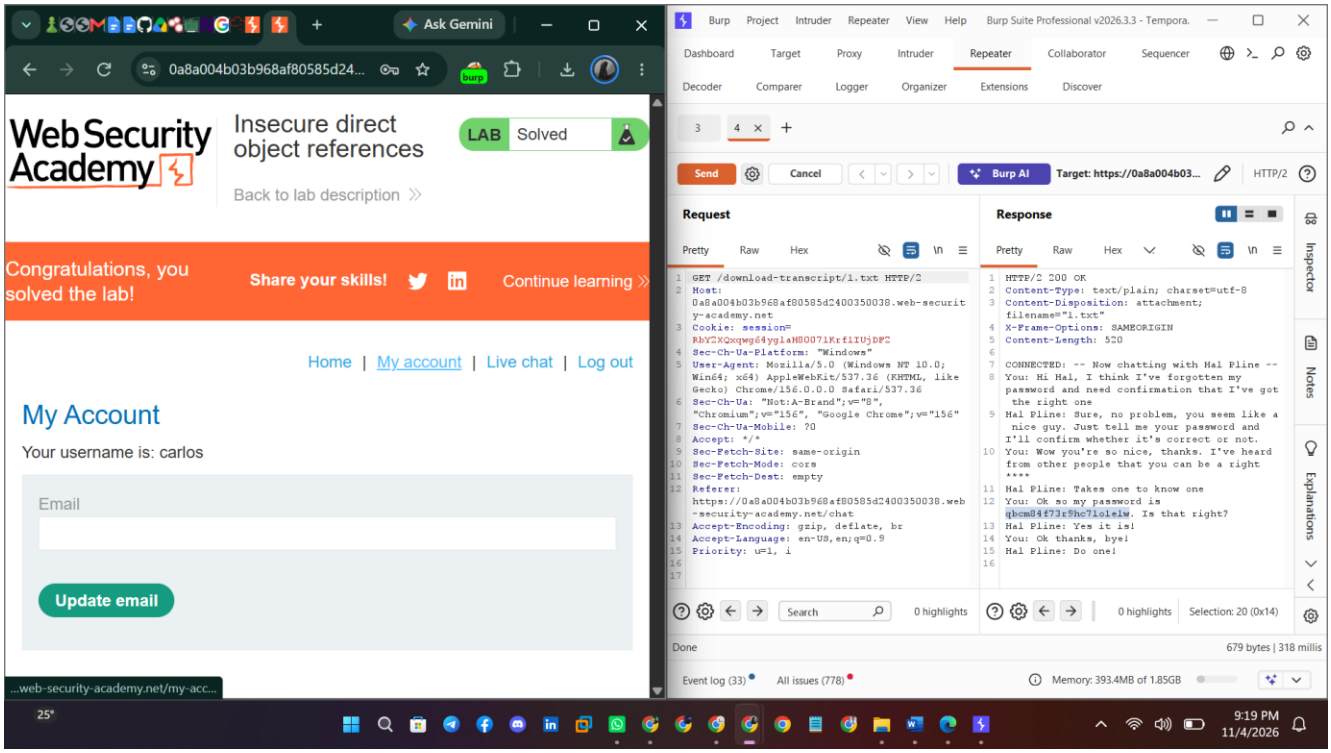


Figure 10: Solve the Lab

6. Result

The lab demonstrated how an insecure transcript-retrieval mechanism could expose another user's private conversation. The disclosed information included credentials that enabled access to the target account, showing how an object-level authorization weakness can escalate into account compromise.

7. Key Learnings

The key lesson was that every resource request must be checked against the identity and permissions of the requester, including requests for static files. Unpredictable filenames may make enumeration harder, but they do not replace access control. Sensitive information should also never be exposed in transcripts or stored in publicly retrievable files.